

Findings Register

Date: 2026-07-20 Status: living document - the single index of everything we have found

Start here. Every finding has a stable ID, a one-paragraph explanation, and a pointer to the detail. Nothing is duplicated: this file says *what* and *why it matters*; the linked documents carry evidence and reproduction.

Findings are split by who owns the fix.

- **THM-n** - the Themis runtime and its APIs. Engineering owns these.
 - **OPS-n** - the operator tooling shipped with it (`nolctl`, service units, the troubleshooting guide). Engineering owns these too, but they are a different component and a different fix.
 - **FW-n** - our validation framework. We own these.
 - **OBS-n** - observations deliberately *not* raised as findings, recorded so they are not rediscovered and re-escalated.
-

Summary

Themis runtime - THM

ID	Finding	Severity	Status
THM-1	A deployed policy has no identity	High	Open, not reported
THM-2	Deployment replaces the entire ruleset	High	Open, not reported
THM-3	Deployment is fire and forget	Medium	Open, not reported
THM-4	Overlapping matches corrupt output (ISSUE-003)	High	Open, handover drafted
THM-5	Replacements truncate at 15 characters (KB-001)	Medium	Open, worked around
THM-6	Evaluation environment unreachable externally	High	Open, not reported
THM-7	No way to check whether the runtime is healthy	Medium	Open, not reported

Operator tooling - OPS

ID	Finding	Severity	Status
OPS-1	<code>nolctl doctor</code> false-fails on kernel parameters	Medium	Open, found 2026-07-20
OPS-2	Service status string never updates after startup	Medium	Open, found 2026-07-20
OPS-3	Troubleshooting guide omits "paused awaiting policy"	Medium	Open, found 2026-07-20

Our framework - FW

ID	Finding	Severity	Status
FW-1	Could report success it had not verified (Tier 0)	Critical	Fixed
FW-2	Blamed Themis for its own expected-output bugs (Tier 1)	Critical	Fixed
FW-3	compare scored unverifiable records as PASS	High	Fixed
FW-4	Transports source a committed config file (Tier 2)	Medium	Open
FW-5	Caller environment silently overridden by config	Medium	Open
FW-6	Failing reports are unusable at scale (Tier 4)	Medium	Open
FW-7	Generation depends on YAML key order (T1-6)	Low	Open

Observations - OBS

ID	Observation	Disposition
OBS-1	Control plane TLS is self-signed	Not a finding. Do not re-promote
OBS-2	Processed payload returns to the caller	Open question , not a defect

THM - Themis runtime

Full detail: [docs/product/themis-product-limitations.md](#) (numbered 1-7 there, matching THM-1 to THM-7).

THM-1 - A deployed policy has no identity

You can post a policy. You cannot ask what policy is loaded. No identifier, no version, no content hash, no deployed-at, no way to read the body back.

Why it matters: during an incident the first question is "what rules were active?" That is unanswerable from the product. It also means a deployed policy cannot be exported, diffed, or restored - if the source file is lost, the enforcing ruleset is unrecoverable.

Felt in practice 2026-07-20: restoring the environment after an outage required the copy we had kept ourselves, because the product could not tell us what had been there.

THM-2 - Deployment replaces the entire ruleset

Posting a policy replaces everything (**persisted**, **REPLACE**). No namespace, no partial update, no rollback, no dry run.

Why it matters: multi-tenancy is unsafe on shared credentials; every change is a full redeploy, which is exactly the operation most likely to introduce the overlapping literals that trigger THM-4; a bad policy is either a DLP outage or silent under-redaction.

THM-3 - Deployment is fire and forget

The deploy call returns on acceptance. `command_id` and `stage: apollo` imply asynchronous distribution, but nothing reports convergence and nothing can be polled.

Why it matters: records sent immediately after deployment may be evaluated against the previous policy. Results can look correct while being produced by the wrong rules - the dangerous case, because nothing indicates it.

THM-4 - Overlapping matches corrupt output

The headline finding. When two rules match overlapping regions of the input, the runtime computes the wrong match start offset and destroys content preceding the match. Silent. HTTP 200 every time.

```
rules    "ABCD" -> "[P]"   and  "DEFG" -> "[Q]"
input    x ABCDEFG y
correct  x [P]EFG y
actual   x [P[Q] y
```

Why it matters: this is ordinary policy authoring, not an exotic edge case. Redacting both `"Acme Corp"` and `"Acme Corporation"` triggers it. Reproduces with curl alone - two rules and one record.

- Detail and evidence: [docs/issues/20260719-ISSUE-003-scale-validation-transformation-mismatch.md](#)
- **Ready to send:** [docs/issues/ISSUE-003-handover-message.md](#)
- Authoring constraint: [docs/issues/KNOWN_BEHAVIORS.md](#)

THM-5 - Replacements truncate at 15 characters

Replacement strings longer than 15 characters are truncated at runtime.

`[FINANCIAL:CREDIT_CARD_NUMBER]` is emitted as `[FINANCIAL:CRED.`

Why it matters: tokens sharing a 15-character prefix become indistinguishable in output, so a reader cannot tell which rule fired. That silently degrades auditability, and it is what made FW-3 possible.

Detail: [docs/issues/KNOWN_BEHAVIORS.md](#) (KB-001).

THM-6 - Evaluation environment unreachable externally

Reachable only inside the VPC, via VPN and in practice an SSH session.

Why it matters: an agent, CI pipeline, customer sandbox, or partner integration cannot establish a VPN or drive SSH. Agent-mediated integration is the fastest-growing category of buyer interest for a product sitting in front of models, and it currently cannot be demonstrated at all.

Framed as demonstrability, not as criticism of the security posture.

THM-7 - No way to check whether the runtime is healthy

No health, readiness, or status endpoint. Verified, not assumed: the data plane host publishes one port, and `/v1/process` is the only route on it - 16 candidate health paths all 404 while `GET /v1/process` returns 405.

Why it matters: the only way to learn the engine is not serving is to send real traffic and have it fail. A long run starts normally and produces nothing but execution failures.

The 2026-07-20 outage is the case study. Apollo boots with its data plane paused and un-pauses only when a policy commits. None had been deployed since the last restart, so every request 503'd for an hour. The system was **one API call from working**, and no signal said so - see OPS-1 to OPS-3 for why the tooling actively misled. A health endpoint reporting *policy-load state*, not just liveness, would have made it a thirty-second fix.

Mitigated our side: `validate run` now pre-flights with one throwaway record and aborts with the remedy rather than generating a full run of failures. It cannot distinguish paused from dead - nothing client-side can - so it names both and leads with the cheap fix.

OPS - Operator tooling

All three found while diagnosing the 2026-07-20 outage. Written up inside THM-7 in `docs/product/themis-product-limitations.md`.

OPS-1 - `nolctl doctor` false-fails on kernel parameters

Reports `FAIL preflight: missing hugepages=4, isolcpus=0-11, nohz_full=0-11, rcu_nocbs=0-11` on a host correctly configured with `hugepages=16, isolcpus=2-13, nohz_full=2-13, rcu_nocbs=2-13`.

It is string-matching one expected topology rather than checking the parameters are present and sane.

Why it matters: it fails on a correct machine and points the operator at GRUB and a reboot - expensive, disruptive, and wrong. It also trains people to ignore doctor output, which defeats the tool.

OPS-2 - Service status string never updates after startup

`systemctl show ares-apollo -p StatusText` still read `data plane PAUSED` after the data plane was verified working. The orchestrator sets the string once at startup and never revises it.

Why it matters: an operator trusting it would restart a healthy service. Combined with OPS-1, the two tools most likely to be consulted both lie.

OPS-3 - Troubleshooting guide omits "paused awaiting policy"

The guide attributes `ARGUS_UPSTREAM_UNAVAILABLE` to "most probably Apollo encountered a severe bug" and prescribes restarting services. For this cause that is wrong: the journal shows apollo restarted three times, each returning to the same paused state.

Why it matters: this is the failure a *new evaluator* hits first, because a fresh or restarted host has no policy yet. Following the guide produces the same 503 and the reasonable conclusion that the product is

broken.

The actual fix, worth adding to the guide verbatim:

```
printf '%s\n' '"SSN" -> "[REDACTED]";' > /tmp/minimal.nol
curl -sS --insecure -X POST "$THEMIS_POLICY_ENDPOINT" \
  -H "Authorization: Bearer $THEMIS_TOKEN" --data-binary @/tmp/minimal.nol
```

FW - Our framework

Detail: [docs/CODE_REVIEW_PLAN.md](#), tiered by risk.

FW-1 - Could report success it had not verified (FIXED)

A 2xx response was treated as success regardless of content, and an empty comparison rendered a green PASS. The framework could certify a product it had never exercised.

Now: success requires a string message in the response; transport exit codes map to categories; an empty comparison renders INCONCLUSIVE.

FW-2 - Blamed Themis for its own bugs (FIXED)

Expected output was computed by scanning only the injected rules, not the full catalog, so 415 of 10,000 documents contained literals the expectation missed - recorded as Themis failures.

Now: full-catalog scanning via Aho-Corasick ([framework/policy/matching.py](#)), and generation refuses catalogs containing nested literals.

FW-3 - [compare](#) scored unverifiable records as PASS (FIXED)

Consequence of THM-5. Normalizing expected replacements to 15 characters is what lets a comparison succeed against a truncating product - but where two replacements share a prefix within that limit, truncation maps them onto the same token, so a record where the *wrong* rule fired compared equal to one where the right rule fired.

Now: colliding tokens are detected, affected would-be passes become **INCONCLUSIVE**, and the report withholds an overall PASS while any exist. Mismatches are deliberately untouched - truncation can only make messages look more alike, so an inequality is still genuine evidence. Commit [ca5c377](#).

Proven closed end to end (2026-07-20). Generation now refuses a catalog whose tokens collapse under truncation, and qualification run [20260720T193444152733Z](#) (5,000 rules / 10,000 records) returned 10,000 PASS with **0 inconclusive** - verified against the manifest and by an independent recount. The prior authoritative run had 4,755 transformations behind collapsed tokens; this one has none.

FW-4 - Transports source a committed config file (OPEN)

Both transports `source config/demo.env`, which is tracked. Anyone who can land a change to it gets code execution plus the tokens sourced next.

FW-5 - Caller environment silently overridden (OPEN)

Scripts source the config *after* the caller's environment, so `THEMIS_ALLOW_INSECURE_TLS=0` on the command line has no effect. Same root cause as transport tests running against a developer's real token.

FW-6 - Failing reports unusable at scale (OPEN)

2.6 MB of undifferentiated blocks, no diff, grouping, or root-cause classification.

FW-7 - Generation depends on YAML key order (OPEN)

Reordering keys in a workload config changes output for a fixed seed. Weakens the determinism guarantee.

OBS - Recorded, deliberately not findings

OBS-1 - Control plane TLS is self-signed

The policy endpoint presents `CN=ip-172-31-40-100`; the processing endpoint has a valid Amazon certificate.

This was briefly written up as a high-severity limitation. That was wrong and it was withdrawn. Do not re-promote it. The sandbox serves one team, is reachable only inside the VPC, and carries no customer traffic. A certificate proves server identity; anyone able to exploit its absence is already inside that network.

The only open question is why the two endpoints differ - and the topology answers it: the data plane is on the local network (10.8.11.x) while the control plane is in EC2 (10.10.1.x). Two provisioning paths, not a decision. Worth one question about production intent, nothing more.

OBS-2 - Processed payload returns to the caller

Today the transformed payload comes back to whoever posted it, who must forward it onward - meaning the caller handles unredacted data on both sides. But the response carries `jid`, `frameId`, and `last: true`, suggesting a framed or streaming protocol we may simply not be using.

Raise as a question, not a finding: is there an inline or proxy mode, and what is the intended production integration pattern? Recorded at the end of the ISSUE-003 handover drafts.

Where things live

Document	Contains
docs/FINDINGS.md	this file - the index of everything
docs/continue-conversation.md	project state, environment, how to resume work
docs/product/themis-product-limitations.md	THM-1 to THM-7 and OPS-1 to OPS-3 in full
docs/issues/20260719-ISSUE-003-*.md	THM-4 evidence and reproduction
docs/issues/ISSUE-003-handover-message.md	ready-to-send Slack and email drafts
docs/issues/KNOWN_BEHAVIORS.md	KB-001 (THM-5) and the THM-4 authoring constraint
docs/CODE_REVIEW_PLAN.md	FW-1 to FW-7 with tiering
docs/issues/technical_debt.md	minor framework debt, no customer impact
docs/architecture/validation-boundaries.md	what the framework does and does not prove
artifacts/evidence/	the policy, failure samples, a reference report

What to send engineering, in order

1. **THM-4 (ISSUE-003)**. Drafts ready. Silent data corruption from ordinary policy authoring - the only finding that destroys customer data.
2. **OPS-1 to OPS-3 plus THM-7**. Cheap fixes, and together they turn a one-call recovery into an hour of misdirected work. Good will costs little.
3. **THM-1 to THM-3**. The policy lifecycle. One root cause: a policy is not a first-class object.
4. **THM-6**. Environment reachability - a commercial argument, not a bug report, and probably a different conversation.

THM-5 is already understood and worked around; raise it as context for THM-4 rather than on its own.